SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Oct – 2026

ASSIGNMENT – 2

Course Code : **CSA43**

Course Name : **INTERNET PROGRAMMING**

Name : **Kandukuri. Kavya**

Registration No : **192425283**

Max Marks**: 50**

Answer all the Questions

1	Develop a JavaScript program using DOM manipulation and Regular Expressions to validate student name, email ID, mobile number and password during online registration. Display appropriate error messages without reloading the webpage.	(10)	(CO2)	(BL3)
2	An e-commerce dynamic product search webpage using JavaScript DOM and AJAX for to Retrieve product details in JSON format from the server and display the matching products dynamically as the user enters a product name.	(10)	(CO2)	(BL4)
3	Discuss the significance of the XMLHttpRequest object, its ready State properties and callback methods. Illustrate the working of AJAX using a suitable JavaScript program .	(10)	(CO2)	(BL3)
4	An engineering college plans to develop a Student Management System using TypeScript to manage student information, including name, register number, department and marks. Develop a TypeScript program using a class and object to represent student details. Include properties, a constructor and a method to display student information.	(10)	(CO2)	(B4)
5	Develop a JavaScript-based form validation system for an online banking application. Use Regular Expressions to validate account number, mobile number and email ID.	(10)	(CO1)	(BL3)

QUESTION 1

Developing a JavaScript Program Using DOM Manipulation and Regular Expressions to Validate Student Registration Details

1. Introduction

Online registration forms are commonly used by educational institutions to collect student information. These forms collect details such as student name, email ID, mobile number and password.

JavaScript is used to validate the information entered by students before submitting the form. DOM manipulation helps JavaScript access and modify HTML elements dynamically, while Regular Expressions (RegEx) are used to check whether the entered information follows a particular format.

In this case study, an online student registration form is developed using HTML, CSS and JavaScript. The form validates student details and displays appropriate error messages without reloading the webpage.

2. Objectives of the Program

The main objectives are:

1. To develop an online student registration form.
2. To understand DOM manipulation using JavaScript.
3. To validate student names using Regular Expressions.
4. To validate email IDs and mobile numbers.
5. To check password strength.
6. To display appropriate error messages for invalid inputs.
7. To prevent form submission when incorrect details are entered.
8. To validate information without reloading the webpage.

3. Introduction to JavaScript

JavaScript is a scripting language used to make web pages interactive and dynamic. It allows developers to respond to user actions, validate forms, display messages and update webpage content.

JavaScript is commonly used with HTML and CSS to develop interactive web applications.

Important features of JavaScript:

- Provides dynamic webpage functionality.
- Supports event handling.
- Performs form validation.
- Allows DOM manipulation.
- Supports Regular Expressions.
- Updates webpage content without reloading.

4. Introduction to DOM Manipulation

DOM stands for Document Object Model. It represents an HTML document as a tree of objects.

JavaScript uses the DOM to access, modify, add or remove HTML elements dynamically.

For example:

document.getElementById("name").value;

This statement accesses the value entered in the HTML element whose ID is name.

Important DOM methods:

DOM Method	**Purpose**
getElementById()	Selects an element using its ID.
querySelector()	Selects the first matching element.
innerHTML	Changes the HTML content.
textContent	Changes the text content.
addEventListener()	Handles user events.
preventDefault()	Prevents the default form action.

DOM manipulation is useful in form validation because it allows error messages to be displayed immediately.

5. Introduction to Regular Expressions

Regular Expressions (RegEx) are patterns used to search, match and validate text.

They help check whether the information entered by a user follows the required format.

Regular Expressions are commonly used to validate:

- Student names.
- Email addresses.
- Mobile numbers.
- Passwords.
- Registration numbers.

JavaScript provides the test() method to check whether a value matches a Regular Expression.

Example:

```
let pattern = /^[A-Za-z]+$/;
let name = "Kavya";


console.log(pattern.test(name));
```

Output:

true

The expression returns true because the name contains only English letters.

6. Validation of Student Name

The student name should contain only alphabetic characters and spaces. It should not contain numbers or special symbols.

The following Regular Expression can be used:

let namePattern = /^[A-Za-z]{3,30}$/;

Explanation:

- ^ represents the beginning of the input.
- [A-Za-z] allows uppercase letters, lowercase letters and spaces.
- {3,30} allows a name between 3 and 30 characters.
- $ represents the end of the input.

This validation prevents students from entering numbers or unwanted special characters in the name field.

7. Validation of Email ID

An email ID is required for communication and registration confirmation. It should contain a valid username, the @ symbol and a domain name.

Regular Expression:

let emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

Explanation:

- [^\s@]+ accepts characters except spaces and @.
- @ checks for the required symbol.
- \. checks for a dot in the domain.
- $ ensures that the input ends correctly.

Example of a valid email:

student@gmail.com

The program displays an error message if the email format is incorrect.

8. Validation of Mobile Number

A mobile number is used to contact students and provide registration updates.

For this program, the mobile number must contain exactly 10 digits and should begin with a digit from 6 to 9.

Regular Expression:

let mobilePattern = /^[6-9][0-9]{9}$/;

Explanation:

- [6-9] checks whether the first digit is between 6 and 9.
- [0-9] allows digits from 0 to 9.
- {9} allows exactly nine additional digits.
- ^ and $ ensure that the entire input matches the pattern.

Example:

9876543210

This validation prevents users from entering letters, special characters or mobile numbers with an incorrect length.

9. Validation of Password

A password protects student account information. Therefore, the registration form should require a strong password.

The password should contain:

- At least 8 characters.
- One uppercase letter.
- One lowercase letter.
- One number.
- One special character.

Regular Expression:

let passwordPattern =

/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$/;

Explanation:

- (?=.*[a-z]) requires a lowercase letter.
- (?=.*[A-Z]) requires an uppercase letter.
- (?=.*\d) requires a number.
- (?=.*[@$!%*?&]) requires a special character.
- {8,} requires a minimum of eight characters.

Example of a valid password:

Student@123

This validation helps students create stronger passwords.

10. Role of DOM Manipulation in Form Validation

DOM manipulation plays an important role in displaying validation results without refreshing the webpage.

When a student clicks the Submit button, JavaScript checks the entered information and displays error messages near the corresponding input fields.

For example:

document.getElementById("error").textContent =

"Please enter a valid email ID";

This statement displays an error message inside the HTML element with the ID error.

The textContent property changes the displayed text dynamically.

11. Event Handling in JavaScript

Event handling allows JavaScript to respond to actions performed by users.

In the registration form, the submit event is used to validate the information when the student clicks the Submit button.

Example:

```
form.addEventListener("submit", function(event) {
   event.preventDefault();
   // Validation code
});
```

Explanation:

- addEventListener() listens for the form submission event.
- submit identifies the event.
- preventDefault() prevents the browser from submitting the form normally.
- JavaScript performs validation without reloading the webpage.

12. Working Principle of the Registration Form

The student registration form follows these steps:

1. The student opens the registration webpage.
2. The student enters their name, email ID, mobile number and password.
3. The student clicks the Submit button.
4. JavaScript retrieves the entered values using DOM methods.
5. Regular Expressions check the format of each input.
6. If any information is incorrect, an appropriate error message is displayed.
7. If all information is valid, a success message is displayed.
8. The webpage does not reload during validation.

13. Error Message Display

Error messages help students identify incorrect information and correct it before submitting the registration form.

Input Field	**Invalid Condition**	**Error Message**
Student Name	Contains numbers or special characters	Please enter a valid name.
Email ID	Incorrect email format	Please enter a valid email ID.
Mobile Number	Not a valid 10-digit number	Please enter a valid mobile number.
Password	Does not satisfy password requirements	Please enter a strong password.

These messages make the registration process easier and more user-friendly.

14. Advantages of DOM and Regular Expression Validation

1. Immediate Validation: Input details can be checked as soon as the form is submitted.
2. Better User Experience: Error messages help users correct mistakes.
3. Reduced Invalid Inputs: Incorrectly formatted information can be identified.
4. No Page Reload: JavaScript validates information without refreshing the webpage.
5. Easy Implementation: Regular Expressions provide a simple way to check input formats.
6. Dynamic Content: DOM manipulation allows error and success messages to appear instantly.
7. Improved Security Awareness: Password rules encourage users to create stronger passwords.

15. Limitations

Although JavaScript validation is useful, it has some limitations.

- Client-side validation can be bypassed by users.
- Regular Expressions may not identify every possible invalid email address.
- Password validation alone cannot guarantee complete account security.
- Server-side validation is also required to protect student information.
- Incorrect Regular Expressions may reject valid inputs.

Therefore, client-side validation should always be supported by server-side validation in real applications.

16. Testing and Evaluation

The registration form should be tested with different input values to ensure that validation works correctly.

Test Case	**Input**	**Expected Result**
Valid Name	Kavya	Accepted
Invalid Name	Kavya123	Error message
Valid Email	kavya@gmail.com	Accepted
Invalid Email	kavya@gmail	Error message
Valid Mobile	9876543210	Accepted
Invalid Mobile	12345	Error message
Valid Password	Student@123	Accepted
Invalid Password	kavya123	Error message

Testing helps identify errors and ensures that the registration form works correctly.

17. Conclusion

JavaScript DOM manipulation and Regular Expressions play an important role in developing interactive online registration forms. DOM methods allow developers to access input fields and display error messages dynamically, while Regular Expressions help validate student names, email IDs, mobile numbers and passwords.

In this case study, an online student registration form is designed to validate user information without reloading the webpage. This improves the user experience, reduces incorrect entries and makes the registration process more efficient.

QUESTION 1: CODE:

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Registration</title>
  <style>
    body {
      font-family: Arial;
      background: #eaf2f8;
      text-align: center;
    }
    .container {
      background: white;
      width: 350px;
      margin: 40px auto;
      padding: 20px;
      border-radius: 10px;
    }
    input {
      width: 90%;
      padding: 10px;
      margin: 8px;
```

```
        }
        button {
            padding: 10px 25px;
            background: #2874a6;
            color: white;
            border: none;
            cursor: pointer;
        }
        .error {
            color: red;
            font-size: 13px;
        }
        #success {
            color: green;
        }
    </style>
</head>
<body>

<div class="container">
    <h2>Student Registration</h2>

    <form id="regForm">
        <input type="text" id="name" placeholder="Student Name">
        <div class="error" id="nameError"></div>

        <input type="text" id="email" placeholder="Email ID">
        <div class="error" id="emailError"></div>

        <input type="text" id="mobile" placeholder="Mobile Number">
        <div class="error" id="mobileError"></div>

        <input type="password" id="password" placeholder="Password">
        <div class="error" id="passwordError"></div>
```

```
        <br><br>
        <button type="submit">Register</button>
    </form>

    <h3 id="success"></h3>
</div>

<script>
document.getElementById("regForm")
.addEventListener("submit", function(event) {

    event.preventDefault();

    let name = document.getElementById("name").value.trim();
    let email = document.getElementById("email").value.trim();
    let mobile = document.getElementById("mobile").value.trim();
    let password = document.getElementById("password").value;

    let valid = true;

    let namePattern = /^[A-Za-z ]{3,30}$/;
    let emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    let mobilePattern = /^[6-9][0-9]{9}$/;
    let passwordPattern =
    /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$/;

    document.querySelectorAll(".error").forEach(e => e.textContent = "");
    document.getElementById("success").textContent = "";

    if (!namePattern.test(name)) {
        document.getElementById("nameError").textContent =
        "Enter a valid name (3-30 letters).";
        valid = false;
```

```
    }

    if (!emailPattern.test(email)) {
        document.getElementById("emailError").textContent =
        "Enter a valid email ID.";
        valid = false;
    }

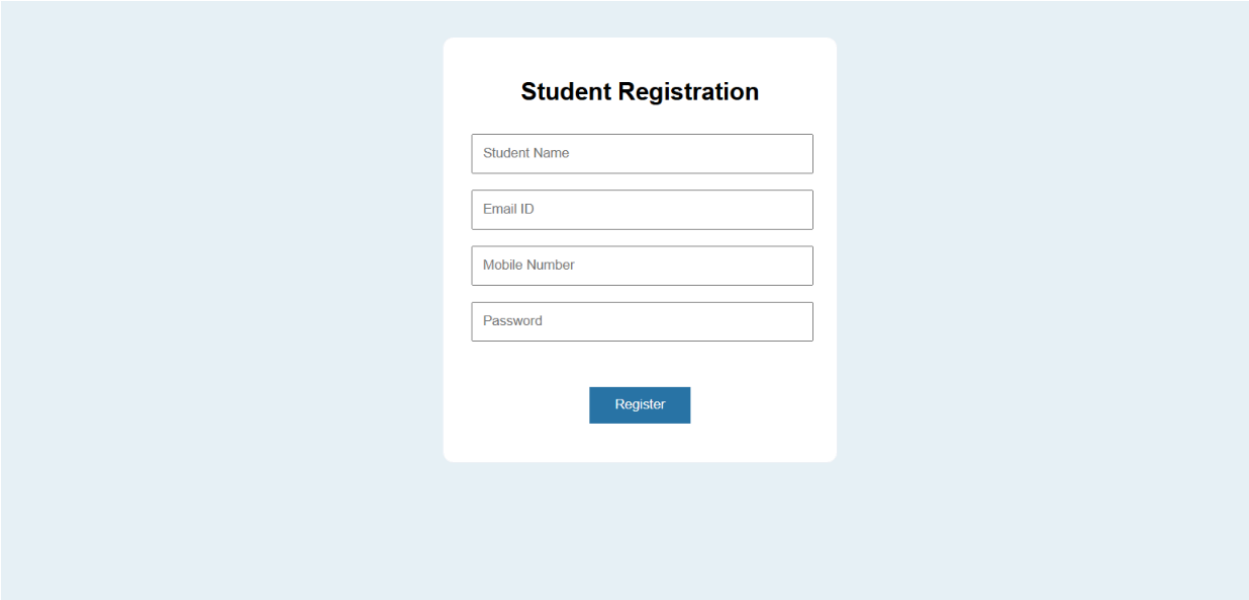
    if (!mobilePattern.test(mobile)) {
        document.getElementById("mobileError").textContent =
        "Enter a valid 10-digit mobile number.";
        valid = false;
    }

    if (!passwordPattern.test(password)) {
        document.getElementById("passwordError").textContent =
        "Use 8 characters with uppercase, lowercase, number and symbol.";
        valid = false;
    }

    if (valid) {
        document.getElementById("success").textContent =
        "Registration Successful!";
        document.getElementById("regForm").reset();
    }
});
</script>

</body>
</html>
```

QUESTION 1: OUTPUT:



QUESTION 2

Developing an E-Commerce Dynamic Product Search Webpage Using JavaScript DOM and AJAX

1. Introduction

E-commerce websites allow customers to search and purchase products online. A dynamic product search system helps users find products quickly by entering a product name in the search box.

JavaScript DOM manipulation and AJAX are used to develop interactive search functionality. AJAX allows a webpage to communicate with a server and retrieve product information without reloading the entire webpage.

In this case study, an e-commerce product search webpage is developed using HTML, CSS, JavaScript, DOM manipulation and AJAX. Product details are retrieved in JSON format and matching products are displayed dynamically as the user enters a product name.

2. Objectives of the Program

The main objectives are:

1. To develop a dynamic e-commerce product search webpage.
2. To understand JavaScript DOM manipulation.
3. To understand the concept of AJAX.
4. To retrieve product details in JSON format.
5. To search products based on the entered product name.
6. To display matching products dynamically.
7. To update search results without reloading the webpage.
8. To improve the user experience of an online shopping website.

3. Introduction to JavaScript

JavaScript is a scripting language used to create interactive and dynamic webpages. It allows websites to respond to user actions and update webpage content.

In an e-commerce website, JavaScript is used to search products, display product details, update prices and handle user interactions.

Important features of JavaScript:

- Makes webpages interactive.
- Supports DOM manipulation.
- Handles user events.
- Communicates with servers using AJAX.
- Processes JSON data.

- Displays dynamic search results.

4. Introduction to DOM Manipulation

DOM stands for Document Object Model. It represents an HTML webpage as a collection of objects that JavaScript can access and modify.

DOM manipulation helps display product details dynamically without changing or reloading the complete webpage.

For example:

```
document.getElementById("results").innerHTML =
"Products Found";
```

This statement changes the content of the HTML element whose ID is results.

Important DOM methods:

DOM Method	Purpose
getElementById()	Selects an HTML element using its ID.
querySelector()	Selects the first matching element.
innerHTML	Inserts or changes HTML content.
textContent	Changes text content.
addEventListener()	Handles user events.
createElement()	Creates a new HTML element.

DOM manipulation is used to display matching products whenever the user enters a search term.

5. Introduction to AJAX

AJAX stands for Asynchronous JavaScript and XML. It is a technique used to exchange data between a webpage and a server without reloading the entire webpage.

Although AJAX originally used XML, modern applications commonly exchange data in JSON format.

In an e-commerce website, AJAX helps retrieve product information from the server whenever a customer searches for a product.

Important features of AJAX:

1. Communicates with the server asynchronously.
2. Updates webpage content without reloading.
3. Retrieves data in JSON format.
4. Improves webpage responsiveness.
5. Reduces unnecessary page loading.

6. Working of AJAX in Product Search

The product search system follows these steps:

1. The user opens the e-commerce webpage.
2. The user enters a product name in the search box.
3. JavaScript captures the search input using an event listener.
4. AJAX sends a request to the server.
5. The server searches for matching products.
6. The server returns product details in JSON format.
7. JavaScript receives and processes the JSON response.
8. DOM manipulation displays the matching products on the webpage.

This process allows users to search for products without refreshing the webpage.

7. Introduction to JSON

JSON stands for JavaScript Object Notation. It is a lightweight data format used to exchange information between a client and a server.

JSON stores information using key-value pairs and is commonly used in modern web applications.

Example of JSON product data:

```
{
  "id": 1,
  "name": "Laptop",
  "price": 45000,
  "category": "Electronics"
}
```

Explanation:

- id represents the product identification number.
- name represents the product name.
- price represents the product price.
- category represents the product category.

JSON makes it easy to transfer product information from the server to the webpage.

8. JavaScript Event Handling

Event handling allows JavaScript to respond to user actions.

In the product search webpage, the input event is used to detect changes whenever the user types something in the search box.

Example:

```
document.getElementById("search")
.addEventListener("input", function() {
    console.log("Searching products...");
```

});

Whenever the user enters or removes a character, the event listener executes the function.

This helps provide dynamic search functionality.

9. Retrieving Product Details Using Fetch API

The Fetch API is a modern JavaScript method used to send HTTP requests and retrieve information from a server.

It can retrieve product information in JSON format.

Example:

```
fetch("products.json")
.then(response => response.json())
.then(data => {
   console.log(data);
});
```

Explanation:

- fetch() sends a request to the specified resource.
- response.json() converts the response into JSON data.
- then() processes the retrieved information.
- data contains the product details.

In this case study, Fetch API is used to demonstrate AJAX-based communication.

10. Dynamic Display of Products

After receiving the JSON data, JavaScript uses DOM manipulation to display matching products.

For example:

```
let result = document.getElementById("results");


result.innerHTML = "<h3>Laptop - ₹45000</h3>";
```

This statement dynamically displays product information inside the webpage.

The search system compares the entered product name with the available product names and displays only matching products.

11. Product Search Functionality

The product search functionality allows customers to find products by entering their names.

For example, when a user enters Laptop, the webpage displays products containing the word Laptop.

Similarly, when the user enters Mobile, the webpage displays matching mobile products.

The search is performed dynamically using JavaScript, AJAX and DOM manipulation.

12. Advantages of Dynamic Product Search

1. Fast Searching: Users can find products quickly.
2. No Page Reload: Search results appear without refreshing the webpage.
3. Better User Experience: Customers can interact with the search box easily.
4. Dynamic Results: Products are displayed based on the entered search term.
5. JSON Support: Product information can be transferred in a simple format.
6. Server Communication: AJAX allows the webpage to retrieve updated product information.
7. Easy Maintenance: Product details can be managed separately on the server.

13. Limitations

- AJAX requires a working server connection.
- Network problems may delay product retrieval.
- Large product databases may require optimized searching.
- Invalid JSON responses may cause errors.
- Server-side security is necessary to protect product information.

14. Testing and Evaluation

The product search webpage can be tested using different search terms.

Test Case	Search Input	Expected Result
Test 1	Laptop	Displays laptop products.
Test 2	Mobile	Displays mobile products.
Test 3	Headphones	Displays headphone products.
Test 4	Watch	Displays watch products.
Test 5	Unknown Product	Displays no products found.
Test 6	Empty Input	Displays all available products.

Testing helps ensure that the dynamic product search system works correctly.

15. Conclusion

JavaScript DOM manipulation and AJAX are important technologies used to develop dynamic e-commerce applications. DOM manipulation helps update webpage content, while AJAX allows the webpage to retrieve product information from the server without reloading the complete page.

In this case study, a dynamic product search webpage is developed to retrieve product details in JSON format and display matching products as the user enters a product name. This makes product searching faster, improves user interaction and provides a better online shopping experience.

QUESTION 2: CODE:

```
<!DOCTYPE html>
<html>
<head>
   <title>Dynamic Product Search</title>
   <style>
      body {
         font-family: Arial;
         background: #eef2f3;
         text-align: center;
      }
      input {
         width: 60%;
         padding: 12px;
         margin: 20px;
      }
      .product {
         background: white;
         padding: 15px;
         margin: 10px auto;
         width: 60%;
         border-radius: 8px;
      }
   </style>
</head>
<body>

   <h2>E-Commerce Product Search</h2>

   <input type="text" id="search"
        placeholder="Enter product name">
```

```
<div id="results">Loading products...</div>

<script>
    let products = [
        {name: "Laptop", price: 45000},
        {name: "Mobile", price: 20000},
        {name: "Headphones", price: 1500},
        {name: "Smart Watch", price: 3000},
        {name: "Keyboard", price: 1000}
    ];

    let blob = new Blob(
        [JSON.stringify(products)],
        {type: "application/json"}
    );

    let url = URL.createObjectURL(blob);

    let productData = [];

    fetch(url)
    .then(response => response.json())
    .then(data => {
        productData = data;
        displayProducts(productData);
    });

    function displayProducts(items) {
        let result = document.getElementById("results");
        result.innerHTML = "";

        if (items.length === 0) {
            result.innerHTML = "No products found";
```

```
            return;
        }

        items.forEach(product => {
            result.innerHTML +=
            `<div class="product">
                <h3>${product.name}</h3>
                <p>Price: ₹${product.price}</p>
            </div>`;
        });
    }

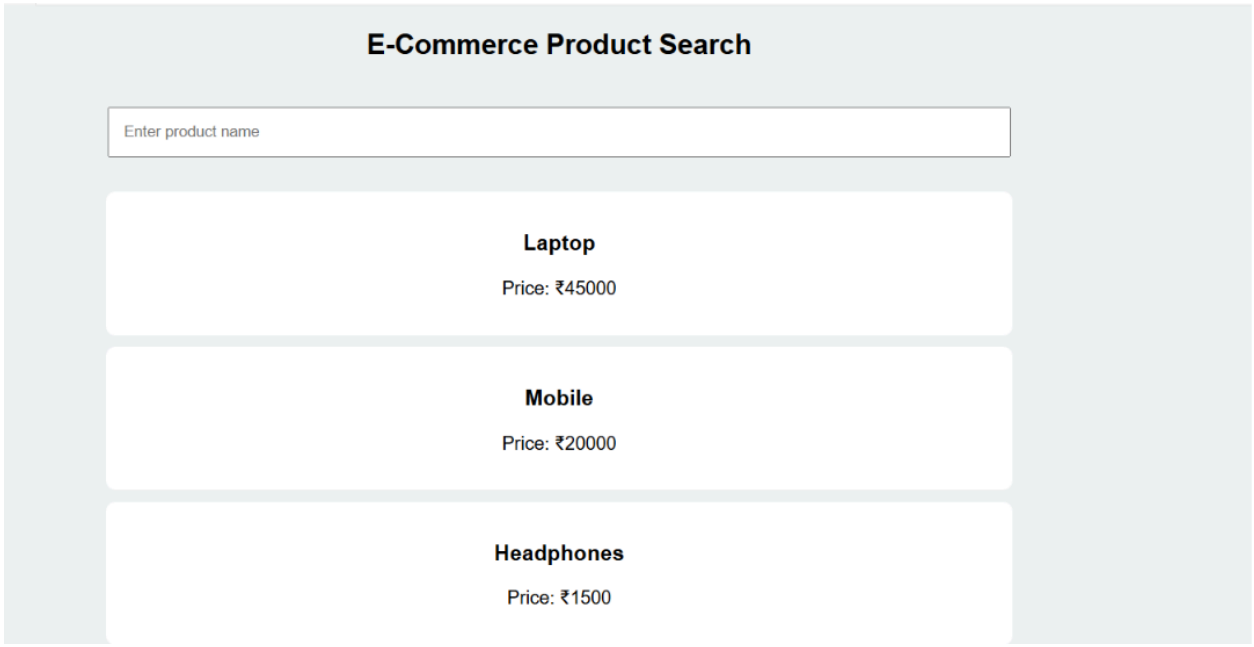
    document.getElementById("search")
    .addEventListener("input", function() {
        let value = this.value.toLowerCase();

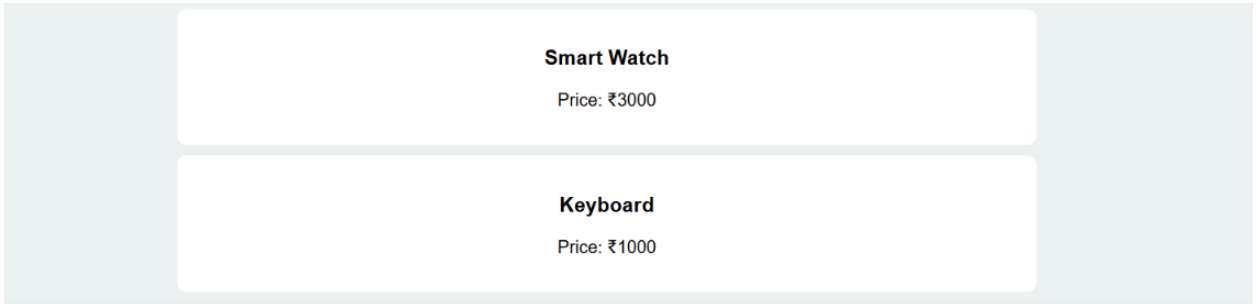
        let filtered = productData.filter(product =>
            product.name.toLowerCase().includes(value)
        );

        displayProducts(filtered);
    });
  </script>

</body>
</html>
```

QUESTION 2: OUTPUT:





QUESTION 3

Discussing the Significance of XMLHttpRequest Object, Its ReadyState Properties and Callback Methods with a Suitable AJAX Program

1. Introduction

AJAX stands for Asynchronous JavaScript and XML. It is a web development technique used to exchange data between a browser and a server without reloading the entire webpage.

The XMLHttpRequest object is an important part of AJAX. It allows JavaScript to send HTTP requests to a server and receive responses asynchronously.

Using AJAX, web applications can update specific parts of a webpage without refreshing the complete page. It improves webpage performance and provides a better user experience.

2. Introduction to XMLHttpRequest Object

The XMLHttpRequest object is a built-in JavaScript object used to communicate with web servers. It helps send HTTP requests and receive data from the server.

It supports different HTTP methods such as GET and POST.

Syntax:

let xhr = new XMLHttpRequest();

This statement creates a new XMLHttpRequest object.

Important methods of XMLHttpRequest:

Method	**Description**
open()	Initializes the request.
send()	Sends the request to the server.
setRequestHeader()	Sets HTTP request headers.
abort()	Cancels the current request.
getResponseHeader()	Retrieves a specific response header.
getAllResponseHeaders()	Retrieves all response headers.

These methods help JavaScript communicate with the server and retrieve information.

3. Significance of XMLHttpRequest Object

The XMLHttpRequest object plays an important role in AJAX-based web applications.

Its significance includes:

1. Asynchronous Communication: It allows the browser to communicate with the server without stopping other webpage activities.
2. No Page Reload: It updates webpage content without refreshing the entire page.

3. Data Retrieval: It retrieves information from the server.
4. HTTP Support: It supports methods such as GET and POST.
5. Dynamic Webpages: It helps develop interactive and responsive applications.
6. Error Handling: It provides information about request failures.
7. Data Exchange: It supports different response formats such as text, XML and JSON.

4. ReadyState Properties of XMLHttpRequest

The readyState property represents the current status of an AJAX request.

It contains five different values, starting from 0 and ending at 4.

ReadyState	Name	Description
0	UNSENT	The request has been created but open() has not been called.
1	OPENED	The open() method has been called.
2	HEADERS_RECEIVED	The request has been sent and response headers have been received.
3	LOADING	The response is being received.
4	DONE	The request has completed successfully or with an error.

Explanation:

- State 0 indicates that the request has not been initialized.
- State 1 indicates that the request has been opened.
- State 2 indicates that the server response headers are available.
- State 3 indicates that response data is being loaded.
- State 4 indicates that the request has finished.

The readyState property is commonly checked using the onreadystatechange callback.

5. Callback Methods in XMLHttpRequest

Callback methods are functions that execute automatically when a particular event occurs during an AJAX request.

They help handle server responses and request errors.

Important callback methods:

a) onreadystatechange

This callback executes whenever the readyState property changes.

Example:

```
xhr.onreadystatechange = function() {
   if (xhr.readyState == 4) {
      console.log("Request completed");
   }
```

};

b) onload

This callback executes when the request completes successfully.

Example:

```
xhr.onload = function() {
   console.log(xhr.responseText);
};
```

c) onerror

This callback executes when a network error occurs.

Example:

```
xhr.onerror = function() {
   console.log("Network Error");
};
```

d) onprogress

This callback executes while the response is being received.

Example:

```
xhr.onprogress = function() {
   console.log("Loading data...");
};
```

e) ontimeout

This callback executes when a request takes longer than the specified timeout.

Example:

```
xhr.ontimeout = function() {
   console.log("Request Timed Out");
};
```

These callback methods help manage different stages of AJAX communication.

6. Working Principle of AJAX

AJAX works through communication between the browser and the web server.

The process follows these steps:

1. The user performs an action on the webpage.
2. JavaScript creates an XMLHttpRequest object.
3. The open() method initializes the HTTP request.
4. The send() method sends the request to the server.
5. The server processes the request.

6. The server sends the response back to the browser.
7. JavaScript receives the response using callback methods.
8. DOM manipulation updates the webpage with the received information.

This process takes place without reloading the entire webpage.

7. HTTP Methods Used in AJAX

HTTP methods specify the type of operation performed when communicating with a server.

GET Method:

The GET method is used to retrieve information from a server.

Example:

```
xhr.open("GET", "data.txt", true);
xhr.send();
```

POST Method:

The POST method is used to send information to a server.

Example:

```
xhr.open("POST", "submit.php", true);
xhr.setRequestHeader(
   "Content-Type",
   "application/x-www-form-urlencoded"
);
xhr.send("name=Kavya");
```

Both methods are commonly used in AJAX applications.

8. Role of DOM Manipulation in AJAX

DOM manipulation is used to display the information received from the server on the webpage.

After receiving a successful response, JavaScript can update an HTML element using innerHTML or textContent.

Example:

```
document.getElementById("result").textContent =
xhr.responseText;
```

This statement displays the server response inside the webpage without refreshing it.

9. Advantages of AJAX

1. Improved Performance: Only the required webpage content is updated.
2. No Page Refresh: AJAX avoids unnecessary webpage reloads.
3. Faster Interaction: Users receive information dynamically.
4. Reduced Server Requests: Only necessary data needs to be transferred.

5. Better User Experience: Webpages become more interactive.
6. Asynchronous Processing: Users can continue interacting with the webpage while data is being retrieved.

10. Applications of AJAX

AJAX is commonly used in many modern web applications.

- Online product search.
- Login and registration forms.
- Live search suggestions.
- Chat applications.
- Email applications.
- Online shopping websites.
- Dynamic news updates.
- Student information systems.

These applications use AJAX to retrieve and display information dynamically.

11. Limitations of AJAX

- AJAX requires a network connection to communicate with a server.
- Network errors may interrupt communication.
- Cross-origin restrictions may prevent requests to certain servers.
- Improper error handling may affect application performance.
- Server-side validation and security are still necessary.

12. Testing and Evaluation

The AJAX program can be tested using the following test cases.

Test Case	**Action**	**Expected Result**
Test 1	Click Load Data	Server data is displayed.
Test 2	Wait for response	Loading message changes to received data.
Test 3	Disconnect the network	Error message is displayed.
Test 4	Click Load Data again	Updated request is processed.

Testing helps verify whether the XMLHttpRequest object communicates correctly with the server.

13. Conclusion

The XMLHttpRequest object is an important component of AJAX technology. It allows JavaScript to communicate with servers and retrieve information without reloading the entire webpage.

The readyState property represents the different stages of an AJAX request, while callback methods such as onreadystatechange, onload and onerror help handle server responses and errors.

In this case study, the working of AJAX is explained using the XMLHttpRequest object. It demonstrates how asynchronous communication and DOM manipulation can be used to develop dynamic and interactive web applications.

QUESTION 3: CODE:

```
<!DOCTYPE html>
<html>
<head>
  <title>AJAX using XMLHttpRequest</title>
  <style>
    body {
      font-family: Arial;
      text-align: center;
      background: #eaf2f8;
    }
    button {
      padding: 10px 20px;
      background: #2874a6;
      color: white;
      border: none;
      cursor: pointer;
    }
    #result {
      margin: 20px;
      padding: 15px;
      background: white;
    }
```

```
    </style>
</head>
<body>

    <h2>AJAX Demonstration</h2>

    <button onclick="loadData()">Load Data</button>

    <div id="result">Click the button to load data</div>

    <script>
        function loadData() {

            let data = "Welcome to AJAX! Data received from server.";

            let blob = new Blob([data], {
                type: "text/plain"
            });

            let url = URL.createObjectURL(blob);

            let xhr = new XMLHttpRequest();

            xhr.onreadystatechange = function() {

                if (xhr.readyState == 4 &&
                    xhr.status == 200) {

                    document.getElementById("result")
                    .textContent = xhr.responseText;
                }
            };

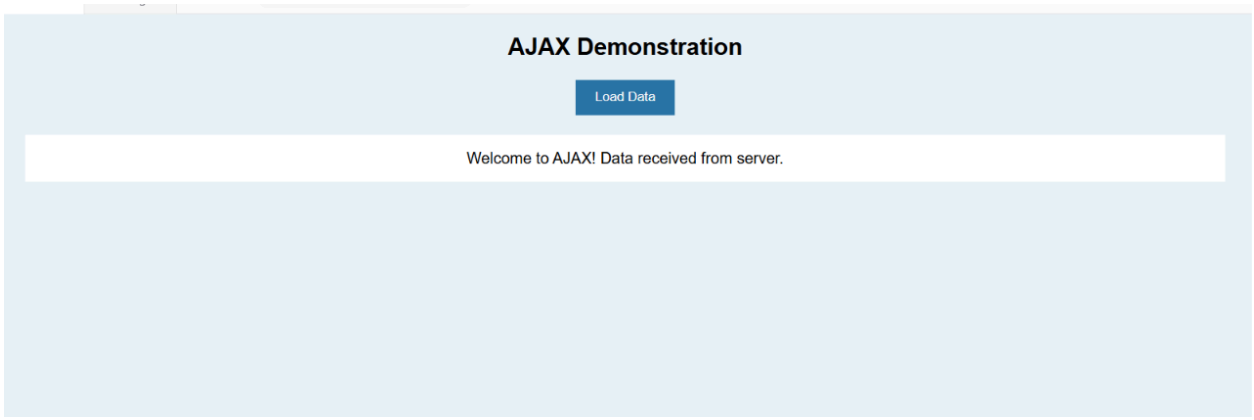
            xhr.onerror = function() {
```

```
            document.getElementById("result")
            .textContent = "Error loading data";
        };

        xhr.open("GET", url, true);
        xhr.send();
    }
  </script>

</body>
</html>
```

QUESTION 3: OUTPUT:



QUESTION 4

Developing a Student Management System Using TypeScript Classes and Objects

1. Introduction

A Student Management System is used by educational institutions to store and manage student information. It helps maintain details such as student name, register number, department and marks.

TypeScript is a programming language developed by Microsoft. It is a superset of JavaScript that supports static typing and object-oriented programming concepts.

In this case study, a simple Student Management System is developed using TypeScript. A class is created to represent student details, and objects are used to store and display student information.

2. Objectives of the Program

The main objectives are:

1. To understand the basics of TypeScript.
2. To develop a simple Student Management System.
3. To understand classes and objects.
4. To declare properties for storing student details.
5. To use a constructor to initialize student information.
6. To create a method to display student details.
7. To understand object-oriented programming concepts.

3. Introduction to TypeScript

TypeScript is an open-source programming language developed by Microsoft. It adds features such as static typing, classes, interfaces and access modifiers to JavaScript.

TypeScript code is converted into JavaScript before it runs in a browser or JavaScript environment.

Important features of TypeScript:

- Supports static type checking.
- Supports classes and objects.
- Provides object-oriented programming.
- Supports inheritance and interfaces.
- Helps identify errors during development.
- Makes large applications easier to maintain.

Example:

```
let studentName: string = "Kavya";
let marks: number = 90;
```

```
console.log(studentName);
console.log(marks);
```

In this example, string specifies the type of student name, and number specifies the type of marks.

4. Introduction to Classes in TypeScript

A class is a blueprint used to create objects. It defines the properties and methods that an object can contain.

In a Student Management System, a class named Student can be created to store student information.

Syntax:

```
class Student {
   name: string;
   registerNumber: string;
   department: string;
   marks: number;
}
```

Here, the Student class contains four properties to store student details.

5. Properties in TypeScript

Properties are variables declared inside a class. They are used to store information about an object.

In the Student Management System, the following properties are used:

Property	Data Type	Description
name	string	Stores the student name.
registerNumber	string	Stores the register number.
department	string	Stores the department name.
marks	number	Stores the student's marks.

Example:

```
name: string;
registerNumber: string;
department: string;
marks: number;
```

These properties represent the basic information of a student.

6. Constructor in TypeScript

A constructor is a special method that is automatically called when an object is created.

It is used to initialize the properties of a class with values.

Syntax:

```
constructor(name: string, marks: number) {
   this.name = name;
   this.marks = marks;
}
```

Explanation:

- constructor defines the constructor method.
- name and marks are parameters.
- this.name refers to the class property.
- The values are assigned when an object is created.

A constructor makes it easier to initialize student information.

7. Methods in TypeScript

A method is a function defined inside a class. It is used to perform a particular operation.

In the Student Management System, a method named displayDetails() is used to display student information.

Example:

```
displayDetails(): void {
   console.log(this.name);
   console.log(this.marks);
}
```

Explanation:

- displayDetails() is the method name.
- void indicates that the method does not return a value.
- console.log() displays the student information.
- this accesses the properties of the current object.

8. Introduction to Objects in TypeScript

An object is an instance of a class. It contains actual values for the properties defined inside the class.

Objects are created using the new keyword.

Example:

```
let student1 = new Student(
   "Kavya",
   "101",
   "Computer Science",
   90
);
```

Here, student1 is an object of the Student class.

The object stores the student's name, register number, department and marks.

9. Working Principle of the Student Management System

The program follows these steps:

1. Create a class named Student.
2. Declare properties for student name, register number, department and marks.
3. Define a constructor to initialize the properties.
4. Create a method named displayDetails().
5. Create student objects using the new keyword.
6. Pass student information through the constructor.
7. Call the display method using the objects.
8. Display the student details on the screen.

This process demonstrates how classes and objects are used to manage student information.

10. Role of Object-Oriented Programming

Object-oriented programming is a programming approach based on classes and objects.

TypeScript supports object-oriented programming, which helps organize code into reusable structures.

Important concepts:

- Class: Defines the structure of student information.
- Object: Represents an individual student.
- Properties: Store student details.
- Constructor: Initializes student information.
- Method: Performs operations such as displaying details.

These concepts make the Student Management System simple and organized.

11. Advantages of Using TypeScript

1. Static Typing: Helps identify data type errors during development.
2. Code Reusability: Classes can be reused to create multiple student objects.
3. Easy Maintenance: Organized code is easier to modify.
4. Object-Oriented Programming: Supports classes and objects.
5. Error Detection: Helps identify mistakes before execution.
6. Readability: Makes programs easier to understand.
7. Scalability: Suitable for developing larger applications.

12. Applications of Student Management System

A Student Management System can be used in educational institutions for:

- Maintaining student records.

- Managing register numbers.
- Storing department details.
- Recording examination marks.
- Displaying student information.
- Organizing academic records.
- Managing student databases.

13. Limitations

- The basic program stores information only during execution.
- It does not permanently save student records.
- It does not include database connectivity.
- It does not provide user authentication.
- Additional features are required for managing large numbers of students.

14. Testing and Evaluation

The program can be tested using different student details.

Test Case	Student Name	Register Number	Department	Marks
Test 1	Kavya	101	CSE	90
Test 2	Nanda Kishore	102	ECE	85
Test 3	Priya	103	IT	95

Testing helps verify whether the class, constructor, objects and display method work correctly.

15. Conclusion

TypeScript classes and objects are useful for developing organized and reusable applications. A class provides a blueprint for storing student information, while objects represent individual students.

In this case study, a Student Management System is developed using a TypeScript class with properties, a constructor and a method to display student information. This demonstrates the use of object-oriented programming in managing student records efficiently.

QUESTION 4 : CODE :

```
class Student {

    name: string;
    registerNumber: string;
    department: string;
    marks: number;

    constructor(name: string, registerNumber: string,
                department: string, marks: number) {

        this.name = name;
        this.registerNumber = registerNumber;
        this.department = department;
        this.marks = marks;
    }

    displayDetails(): void {
        console.log("Student Name: " + this.name);
        console.log("Register Number: " + this.registerNumber);
        console.log("Department: " + this.department);
        console.log("Marks: " + this.marks);
        console.log("----------------------");
    }
}

// Creating student objects

let student1 = new Student(
    "Kavya", "101", "Computer Science", 90
```

```
);

let student2 = new Student(
    "Rahul", "102", "Electronics", 85
);

// Displaying student information

student1.displayDetails();
student2.displayDetails();
```

QUESTION 4: OUTPUT:

```
class Student { name: string; registerNumber: string; department: string; marks: number; constructor(name: string, registerNumber: string, department: string, marks: number) { this.name = name; this.registerNumber = registerNumber; this.department = department; this.marks = marks; } displayDetails(): void { console.log("Student Name: " + this.name); console.log("Register Number: " + this.registerNumber); console.log("Department: " + this.department); console.log("Marks: " + this.marks); console.log("----------"); } } // Creating student objects let student1 = new Student( "Kavya", "101", "Computer Science", 90 ); let student2 = new Student( "Rahul", "102", "Electronics", 85 ); // Displaying student information student1.displayDetails(); student2.displayDetails();
```

QUESTION 5

Developing a JavaScript-Based Form Validation System for an Online Banking Application Using Regular Expressions

1. Introduction

Online banking applications allow customers to perform financial activities such as checking account balances, transferring money and managing their accounts through the internet.

During online banking registration, customers must provide important details such as account number, mobile number and email ID. These details must be validated to ensure that users enter information in the correct format.

JavaScript and Regular Expressions are used to validate the information entered into online banking forms. JavaScript checks the user input, while Regular Expressions identify whether the entered details follow the required format.

In this case study, a JavaScript-based form validation system is developed for an online banking application. It validates the account number, mobile number and email ID and displays appropriate error messages without reloading the webpage.

2. Objectives of the Program

The main objectives are:

1. To develop an online banking form.
2. To understand JavaScript form validation.
3. To use Regular Expressions for validating user information.
4. To validate bank account numbers.
5. To validate mobile numbers.
6. To validate email addresses.
7. To display appropriate error messages.
8. To prevent form submission when invalid information is entered.
9. To improve the accuracy of customer information.

3. Introduction to JavaScript

JavaScript is a scripting language used to create interactive and dynamic webpages. It allows developers to validate user inputs, handle events and update webpage content.

In online banking applications, JavaScript helps validate customer information before submitting forms.

Important features of JavaScript:

- Supports form validation.
- Provides dynamic webpage functionality.
- Handles user events.
- Supports DOM manipulation.
- Works with Regular Expressions.
- Displays error and success messages.

Example:

```
let account = "123456789012";
console.log(account);
```

This code stores an account number in a JavaScript variable.

4. Introduction to Regular Expressions

Regular Expressions (RegEx) are patterns used to match and validate text.

They are useful for checking whether the information entered by customers follows a particular format.

Regular Expressions can be used to validate:

- Bank account numbers.
- Mobile numbers.
- Email addresses.
- Passwords.
- Customer identification details.

JavaScript provides the test() method to check whether an input matches a particular pattern.

Example:

```
let pattern = /^[0-9]+$/;
let account = "1234567890";

console.log(pattern.test(account));
```

Output:

true

The result is true because the account number contains only digits.

5. Validation of Bank Account Number

An account number is an important identification detail used in banking transactions.

In this program, the account number must contain exactly 12 digits.

Regular Expression:

```
let accountPattern = /^[0-9]{12}$/;
```

Explanation:

- ^ represents the beginning of the input.
- [0-9] allows digits from 0 to 9.
- {12} specifies exactly 12 digits.
- $ represents the end of the input.

Example of a valid account number:

123456789012

This validation prevents customers from entering alphabets, special characters or account numbers with an incorrect length.

Note: The 12-digit rule is used as an example for this assignment. Actual account number lengths vary between banks.

6. Validation of Mobile Number

A mobile number is required for customer communication, transaction alerts and banking notifications.

For this program, the mobile number must contain exactly 10 digits and begin with a number between 6 and 9.

Regular Expression:

let mobilePattern = /^[6-9][0-9]{9}$/;

Explanation:

- [6-9] checks the first digit.
- [0-9] allows the remaining digits.
- {9} specifies nine additional digits.
- ^ and $ ensure that the complete input matches the pattern.

Example:

9876543210

This validation ensures that the customer enters a properly formatted mobile number.

7. Validation of Email ID

An email ID is used by banks to communicate important information such as account notifications and transaction details.

A valid email address should contain a username, the @ symbol and a domain name.

Regular Expression:

let emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

Explanation:

- [^\s@]+ accepts characters except spaces and @.
- @ checks for the required symbol.

- \. checks for a dot in the domain.
- $ ensures that the input ends correctly.

Example of a valid email:

customer@gmail.com

This validation helps prevent customers from entering incorrectly formatted email addresses.

8. DOM Manipulation in Banking Form Validation

DOM stands for Document Object Model. It allows JavaScript to access and modify HTML elements dynamically.

DOM manipulation is used to retrieve customer details and display validation messages.

Example:

```
document.getElementById("error").textContent =
"Enter a valid account number";
```

This statement displays an error message inside the HTML element with the ID error.

Important DOM methods:

Method	**Purpose**
getElementById()	Selects an HTML element.
textContent	Changes the text content.
addEventListener()	Handles user events.
preventDefault()	Prevents normal form submission.
reset()	Clears the form fields.

These methods help create an interactive banking form.

9. Event Handling in JavaScript

Event handling allows JavaScript to respond to user actions.

In the online banking form, the submit event is used to validate customer details when the Submit button is clicked.

Example:

```
form.addEventListener("submit", function(event) {
   event.preventDefault();
   // Validation code
});
```

Explanation:

- addEventListener() listens for form submission.
- submit identifies the event.
- preventDefault() prevents the webpage from submitting normally.

- JavaScript checks the entered information.

This allows validation without reloading the webpage.

10. Working Principle of the Banking Form Validation System

The banking form validation system follows these steps:

1. The customer opens the online banking webpage.
2. The customer enters the account number.
3. The customer enters the mobile number.
4. The customer enters the email ID.
5. The customer clicks the Submit button.
6. JavaScript retrieves the entered details using DOM methods.
7. Regular Expressions validate each input.
8. If any information is incorrect, an error message is displayed.
9. If all details are valid, a success message is displayed.
10. The form does not reload during validation.

This process helps maintain correctly formatted customer information.

11. Error Message Display

Error messages help customers identify incorrect information and correct it before submitting the banking form.

Input Field	Invalid Condition	Error Message
Account Number	Not exactly 12 digits	Please enter a valid 12-digit account number.
Mobile Number	Invalid 10-digit number	Please enter a valid mobile number.
Email ID	Incorrect email format	Please enter a valid email ID.

These messages make the banking registration process easier and more user-friendly.

12. Advantages of Regular Expression Validation

1. Accurate Input: Helps identify incorrectly formatted information.
2. Fast Validation: Checks customer details immediately.
3. Better User Experience: Displays useful error messages.
4. Reduced Errors: Prevents common input mistakes.
5. Easy Implementation: Regular Expressions provide simple validation patterns.
6. Dynamic Updates: DOM manipulation displays results without reloading the webpage.
7. Improved Data Quality: Helps maintain correctly formatted customer information.

13. Limitations

- Client-side validation can be bypassed.
- Regular Expressions cannot confirm whether an account number actually exists.

- A valid mobile number format does not prove ownership of the number.
- A valid email format does not confirm that the email address exists.
- Server-side validation is required for real banking applications.

For security, real online banking systems must also use secure authentication, encrypted communication and server-side verification.

14. Testing and Evaluation

The banking form can be tested using different input values.

Test Case	Input	Expected Result
Test 1	123456789012	Accepted
Test 2	12345	Invalid account number
Test 3	9876543210	Accepted
Test 4	1234567890	Invalid mobile number
Test 5	customer@gmail.com	Accepted
Test 6	customer@gmail	Invalid email ID

Testing helps verify whether the banking form validates customer information correctly.

15. Conclusion

JavaScript and Regular Expressions are useful technologies for developing online banking form validation systems. Regular Expressions help validate account numbers, mobile numbers and email IDs, while JavaScript DOM manipulation displays appropriate error messages dynamically.

In this case study, an online banking form validation system is developed to check customer information without reloading the webpage. This improves input accuracy, reduces common errors and provides a better user experience. However, real banking applications must also use server-side validation and proper security mechanisms to protect customer information.

QUESTION 5: CODE:

```
<!DOCTYPE html>
<html>
<head>
   <title>Online Banking Form</title>

   <style>
      body {
```

```
    font-family: Arial;
    background: #eaf2f8;
    text-align: center;
}

.container {
    background: white;
    width: 350px;
    margin: 40px auto;
    padding: 20px;
    border-radius: 10px;
}

input {
    width: 90%;
    padding: 10px;
    margin: 8px;
}

button {
    padding: 10px 25px;
    background: #2874a6;
    color: white;
    border: none;
    cursor: pointer;
}

.error {
    color: red;
    font-size: 13px;
}

#success {
    color: green;
```

```
        }
    </style>
</head>

<body>

<div class="container">

    <h2>Online Banking Registration</h2>

    <form id="bankForm">

        <input type="text" id="account"
               placeholder="Account Number">
        <div class="error" id="accountError"></div>

        <input type="text" id="mobile"
               placeholder="Mobile Number">
        <div class="error" id="mobileError"></div>

        <input type="text" id="email"
               placeholder="Email ID">
        <div class="error" id="emailError"></div>

        <br><br>

        <button type="submit">Submit</button>

    </form>

    <h3 id="success"></h3>

</div>
```

```
<script>

document.getElementById("bankForm")
.addEventListener("submit", function(event) {

    event.preventDefault();

    let account = document.getElementById("account").value.trim();
    let mobile = document.getElementById("mobile").value.trim();
    let email = document.getElementById("email").value.trim();

    let accountPattern = /^[0-9]{12}$/;
    let mobilePattern = /^[6-9][0-9]{9}$/;
    let emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

    let valid = true;

    document.querySelectorAll(".error")
    .forEach(e => e.textContent = "");

    document.getElementById("success").textContent = "";

    if (!accountPattern.test(account)) {
        document.getElementById("accountError")
        .textContent = "Enter a valid 12-digit account number.";
        valid = false;
    }

    if (!mobilePattern.test(mobile)) {
        document.getElementById("mobileError")
        .textContent = "Enter a valid mobile number.";
        valid = false;
    }
```

```
    if (!emailPattern.test(email)) {
        document.getElementById("emailError")
        .textContent = "Enter a valid email ID.";
        valid = false;
    }

    if (valid) {
        document.getElementById("success")
        .textContent = "Banking Details Submitted Successfully!";

        document.getElementById("bankForm").reset();
    }

});

</script>

</body>
</html>
```

QUESTION 5: OUTPUT :

